

Module 0: Tools — VS Code, a C++ Compiler, Git & GitHub

Introduction to C++ Programming — A Beginner's Course

Module Overview

This module is entirely logistical, and that is exactly why it deserves a full session rather than being rushed as "homework before we start." Every student needs a working compiler, a comfortable editor, and a GitHub account signed into VS Code before Module 1 begins — a single broken install here costs far more class time later than doing it right now, together, in the room.

Suggested Duration: 45–60 minutes (can run longer for a class with mixed OS/hardware; budget extra time if any students are on Windows or shared/locked-down lab machines)

Learning Objectives

By the end of this module, students will be able to:

- Install and verify a working C++ compiler (g++ via Xcode Command Line Tools on macOS; MinGW or WSL on Windows).
- Install VS Code and the C/C++ and Code Runner extensions.
- Compile and run a first program from the terminal, by hand, understanding each part of the command.
- Create a GitHub account and sign into it from VS Code.
- Execute the full status → add → commit → push workflow at least once, from both the terminal and the VS Code Source Control panel.

Prerequisites

- None. This is the first session — treat every student as a first-time user of a terminal.

Materials & Instructor Prep

- A projector or screen-share so students can follow each terminal command visually, not just by ear.
- A shared document or slide with every command used in this session, so students can copy-paste-verify rather than mistype during setup (typos in setup are pure friction, not learning).

- Have your own GitHub account and a demo repository ready to project as a live example before students touch their own.

Key Vocabulary

Term	Definition
Compiler	A program that translates human-readable source code into an executable program the computer can run.
Terminal / Shell	A text-based interface for typing commands directly to the operating system.
IDE / Editor	A program for writing code with helpful features like syntax highlighting and error detection (VS Code, in this course).
Git	A tool that tracks changes to files over time, on the student's own machine.
GitHub	A website that hosts a copy of a Git history online, for backup and sharing.
Repository ("repo")	A folder that Git is tracking the history of.
Commit	A saved snapshot of the repo at a point in time, with a message describing what changed.

Lesson Outline & Instructor Talking Points

1. Frame why this matters before touching a keyboard (5 min)

- Open with the pipeline diagram: source code → compiler → executable. Say explicitly: "Nothing you type today will 'just run' the way a script does in some other languages — there is always a translation step, and that's the compiler."
- Name the destination early: "By the end of this course, you'll have a real project on your own GitHub profile — something you can link to on a CV. Git and GitHub are not busywork; they are the single most transferable habit in this course."

Instructor note: Resist the urge to explain Git's internal model (snapshots, DAGs, etc.) today. Students only need the four-command loop (`status/add/commit/push`) to function — depth comes later through repetition, not a lecture.

2. Install the compiler, live, together (10–15 min)

- Project your own terminal. Run `xcode-select --install` (macOS) and narrate what's happening while it installs.

- Have everyone run `g++ --version` and hold up their screen or say 'done' in chat/aloud once they see a version number.
- For any Windows students, this is the point to switch to your platform-specific fallback (MinGW installer or WSL) — decide which one in advance and don't improvise live.

Instructor note: *This step has the highest variance in a mixed-device classroom. Start it first, since installs can run in the background while you talk through the next section.*

3. Install VS Code and extensions (10 min)

- Download, install, open. Show the Extensions icon and search for 'C/C++' live — point out how to tell an official Microsoft extension from a look-alike (checkmark badge, install count).
- Install Code Runner and demonstrate the ► button once, but immediately say: "We're using the terminal method first on purpose — the button hides what's actually happening, and you need to see it at least once."

4. Compile and run hello.cpp by hand (10 min)

- Type the file live, character by character, narrating `#include`, `main`, `cout`, the semicolon.
- Run `g++ hello.cpp -o hello` and immediately ask: "What do you think the `-o hello` part means?" before explaining — a two-second guess primes retention better than a straight explanation.
- Run `./hello`. Celebrate this — it is a real, if tiny, working program.

Instructor note: *Deliberately misspell something (e.g. `Cout` instead of `cout`) in a second pass and show the resulting compiler error. Naming the error message out loud, once, demystifies the wall of red text every beginner will otherwise panic at later.*

5. GitHub account + VS Code sign-in (10 min)

- Everyone creates a GitHub account now, in class, with a professional username — flag that this username may end up on a CV someday.
- Sign into GitHub from VS Code's Accounts icon. Confirm as a class that everyone sees their username in the corner.

6. The status → add → commit → push loop (10 min)

- Run `git status` and ask students to describe in their own words what it's telling them, before you explain.
- Stage, commit, and push `hello.cpp` as a class, once via terminal commands and once via the Source Control panel's +/checkmark/sync icons — show both because different students will gravitate to different tools.

- Have every student confirm their file appears on GitHub.com in their browser before moving on. Do not skip this verification step.

Instructor note: *This is the single most common place setup silently fails (auth issues, wrong folder, forgetting to stage). Walk the room; do not assume the group screen-share means everyone succeeded.*

Common Student Misconceptions

Misconception	How to Address It
"The compiler and the editor are the same thing."	Point out VS Code can open any text file and never compiles anything by itself — g++ (or the extension calling it) is doing all the real work.
"Once I install Git, my code is automatically backed up."	Emphasize that nothing is saved to Git history until commit, and nothing reaches GitHub until push — both are deliberate actions, not automatic.
"A compiler error means I broke something serious."	Reframe: an error means the compiler caught a mistake before it could run — that's a safety net, not a failure.

Exercises & Module Project

There is no separate project this module — the module's own checkpoint IS the deliverable: a working hello.cpp compiled from the terminal, committed, and pushed to a personal GitHub repository.

Facilitation guidance: Walk through the checklist in 00-setup/README.md as a class and do not let anyone leave the room (or session) with an unchecked box. This module has zero conceptual difficulty and 100% environmental friction — the entire point is removing that friction now, once, so it never blocks a lesson again.

Discussion & Comprehension Check Questions

1. In your own words, what does the compiler actually do to your text file?

Guidance: Look for: translates human-readable code into something the machine can execute; happens before the program runs, not while it runs.

2. What is the difference between committing and pushing?

Guidance: Commit = save a snapshot locally; push = upload that snapshot (and its history) to GitHub.

Assessment Ideas

- No formal quiz — use the setup checklist itself as a pass/fail checkpoint.
- Optional exit ticket: "Write the exact four commands, in order, you'd run to save and back up a change you just made."

Answer Key Notes

There is no exercises/solutions folder for this module. The 'answer' is simply a green checkmark on every box in the Module 0 checklist and a visible commit on the student's GitHub profile.

Wrap-Up & Bridge to Next Module

Close by pointing at the roadmap: "Everything from here is C++ itself — the annoying one-time setup is over." Confirm every student has pushed at least one commit before ending the session.

Instructor Tips

- Do this module live and synchronously if at all possible — it is the worst module to assign as unsupervised homework, because a stuck student here has no way to ask a clarifying question mid-install.
- Keep a short list of platform-specific fallbacks ready (Windows/WSL, locked-down lab machines, Chromebooks) before class starts, not during.
- Bank 10 extra minutes of slack in this module specifically — it always runs long the first time.